In C++, we used local and global variables — but how does Java manage variables and their scope within a class?

Variables in Java

By: Bilal Ahmed

Types of variables in Java

Local Variable

Instance Variable

Class/Static Variable

Local Variable

- A **local variable** is declared inside a method or block and can be used only within that method.
- Used for temporary data needed only while the method runs.

```
public class Test {
   public static void main(String[] args) {

      int age = 20;   // Local variable
      System.out.println(age);
   }
}
```

Use cases

- Loop counters
- Temporary calculations
- User input inside a method

Instance Variable

- An **instance variable** is declared inside a class but outside methods.
- Each object of the class has its own copy.
- Stores data specific to each object (e.g., each student's name).

```
public class Test {

   String name = "Ali";   // Instance variable

   public static void main(String[] args) {
      Test obj = new Test();
      System.out.println(obj.name);
   }
}
```

Use cases

STUDENT NAME, AGE, MARKS

BANK ACCOUNT BALANCE

EMPLOYEE DETAILS

Static (Class) Variables

- A **static variable** is declared using the static keyword and is shared among all objects of the class.
- Stores common data shared by everyone (e.g., same university for all students).

```
public class Test {

   static String university = "ABC University";   //
Static variable

   public static void main(String[] args) {
      System.out.println(university);
   }
}
```

Use cases

University name for all students

Company name for all employees

Counter for total objects created

Quick Summary

Local → Inside method → Temporary

Instance → Per object → Unique data

Static → Per class → Shared data

Memory Management in Java

Stack are used for local variables.

Every method has its own stack.

Static and instance variables use heap.

Stack Memory

- **Purpose:** Stores **local variables** and **method call information**.
- **Characteristics:**
  - LIFO (Last In, First Out)
  - When a method finishes, its local variables are automatically removed.
  - Very fast to access.

Reason stack is used: local variables are temporary and short-lived, so stack allows **automatic cleanup**.

Heap Memory

- **Purpose:** Stores **objects** and **class/instance data**.
- **Characteristics:**
  - Used for **dynamic memory allocation**.
  - Slower than stack but flexible
  - Objects stay in memory until no references exist (garbage collection).

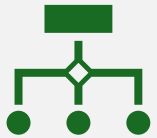
Reason heap is used: instance/static data must **persist beyond a single method call**, unlike local variables.

Comparison

Memory Type	Stores	Lifetime	Access Speed	Cleanup
Stack	Local variables, method calls	Until method exits	Fast	Automatic
Heap	Objects, instance/static variables	Until no reference exists (GC)	Slower	Garbage Collector

Lifetime & Scope

Local variables exist only during the **method call** → perfect for stack.

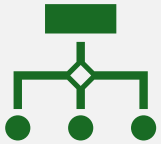


Instance variables exist as long as the **object exists** → need heap.

Static variables exist as long as the **class is loaded** → heap (method area in JVM).

Access Mechanism

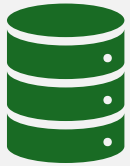
Stack variables are **accessed via offsets** → very fast.



Heap objects are accessed via **references/pointers** → slightly slower.

Memory Management

Stack is **automatically managed** → no manual cleanup.



Heap requires **garbage collection** → JVM tracks objects no longer referenced.

Size & Limitations

Stack is **smaller** → deep recursion may cause **StackOverflowError**.

Heap is **larger** → can store many objects but excessive use can cause **OutOfMemoryError**.

Summary

STACK → TEMPORARY, FAST, AUTO-CLEANED → LOCAL VARIABLES

HEAP → LONG-LIVED, FLEXIBLE → OBJECTS & CLASS DATA (INSTANCE/STATIC)

Example

- k is declared **inside main** → it is a **local variable**, so its **scope is limited to main only**.
- display() is an **instance method**, trying to access k → **cannot see local variables of another method**.
- **Result:** Compilation error: cannot find symbol: variable k.

```
class test {
    public static void main(String[]
args){
        int k = 10;
        }
        void display(){
            System.out.println(k);
    }
}
```

Example

```
class test {
    int k=10;  // declaration
    k = 15;
    public static void main(String[] args) {
}
}
```

Explanation

- int k = 10; — this is **declaration + initialization** of an **instance variable**.
- k = 15; — **cannot have standalone statements directly in the class body** outside a method, constructor, or initializer block.
- **Result:** Compilation error: illegal start of expression.

Solution: Use Constructor

```
class test {
   int k = 10;  // declaration
   test() {
      k = 15;  // assign in constructor
   }
   public static void main(String[] args) {
      test t = new test();
      System.out.println(t.k); // prints 15
   }
}
```

Example

```
class test {
    int k = 10;
    public static void main(String[] args) {
    display();
    }
     void display(){
        k = 7;
        System.out.println(k);
    }
}
```

Output Clear

```
ERROR!
/tmp/2Cfw858Zsf/Main.java:5: error: non-static method
    display() cannot be referenced from a static context
    display();
    ^
1 error
ERROR!
error: compilation failed
```

Explanation

display() is an **instance method** (non-static).

main() is **static**, which belongs to the **class, not any object**.

Static methods cannot directly call non-static methods or access non-static variables.

Result: Compilation error: non-static method display() cannot be referenced from a static context.

Solution

```
class test {
    int k = 10;
    // k = 15;
    public static void main(String[] args) {
    test dis = new test();
    dis.display();
    }
     void display(){
        k = 7;
        System.out.println(k);
    }
}
```

Data Types in Java

Primitive Data Types

Built-in data types provided by Java.

Store **actual values** directly in memory.

Faster and memory efficient.

Primitive Data Types

Type	Example
int	10
float	3.14f
double	3.14159
char	'A'
boolean	true / false
byte	100
short	2000
long	99999

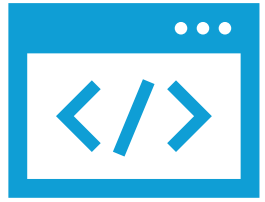
Example

```
public class Test {
    public static void main(String[] args) {

        int age = 21;
        double cgpa = 3.75;
        char grade = 'A';
        boolean passed = true;

        System.out.println(age);
        System.out.println(cgpa);
        System.out.println(grade);
        System.out.println(passed);
    }
}
```

Non-Primitive Data Types

Also called **Reference Types**.

Store **reference (address)** of data, not actual value.



Created by programmer or provided by Java.

Non-Primitive Data Types

String

Arrays

Classes

Objects

Interfaces

Primitive vs Non-Primitive

Feature	Primitive	Non-Primitive
Stores	Actual value	Reference
Memory	Fixed	Flexible
Speed	Faster	Slightly slower
Examples	int, char	String, Array

Summary

PRIMITIVE → SIMPLE BUILT-IN TYPES

NON-PRIMITIVE → COMPLEX REFERENCE TYPES

BOTH ARE USED TO STORE DATA IN VARIABLES

Declaration and Initialization

```
class test {
    public static void main(String[] args) {

        int a = 10;   // Declaration + Initialization

        System.out.println(a);
    }
}
```

```
class test {
    public static void main(String[] args) {
        int b;   // Only Declaration
        b = 5;   // Initialization later

        System.out.println(b);
    }
}
```

Using Variable Without Initialization

- **Output (error):** variable x might not have been initialized
- Java does not allow use of uninitialized local variables.

```
class test {
    public static void main(String[] args) {
        int x;
        System.out.println(x);   // Compile-time
Error
    }
}
```

In C++

- **Output (often):** 0
- Not guaranteed
- Could be garbage value
- Depends on memory state

```
#include <iostream>
using namespace std;

int main() {
    int x;
    cout << x;
}
```

Java vs C++ Behavior

Language	Uninitialized Local Variable
C++	Allowed (garbage/0/random)
Java	Compile-time error ❌

Java is Strict

- **Java prevents:**
  - Garbage values
  - Logical bugs
  - Security risks
  - Unpredictable outputs
- **Design Goal:**
  - Reliable and safe programs.

Important Clarification

Variable Type	Default Value Example
Instance	0, null, false
Static	0, null, false
Local	❌ No default value

Why Only Local Variables?

Variable Type	**Memory Area**
Local	Stack
Instance	Heap
Static	Method Area

JVM Responsibility (Instance / Static)

- JVM allocates memory
- JVM initializes defaults
- JVM manages lifecycle
- Safe without manual initialization

Local Variables

Created inside methods

Stored on stack

Method-controlled lifecycle

JVM does not auto-initialize

Must be initialized by programmer

Performance Reason

If Java initialized locals automatically:

- Extra work on every method call
- Slower execution
- Unnecessary memory operations

So Java keeps locals:

- Fast
- Lightweight
- Programmer-controlled

Simple Comparison Analogy

Instance/Static: Pre-furnished hostel rooms (ready to use)

Local: Empty classroom desk (arrange before use)